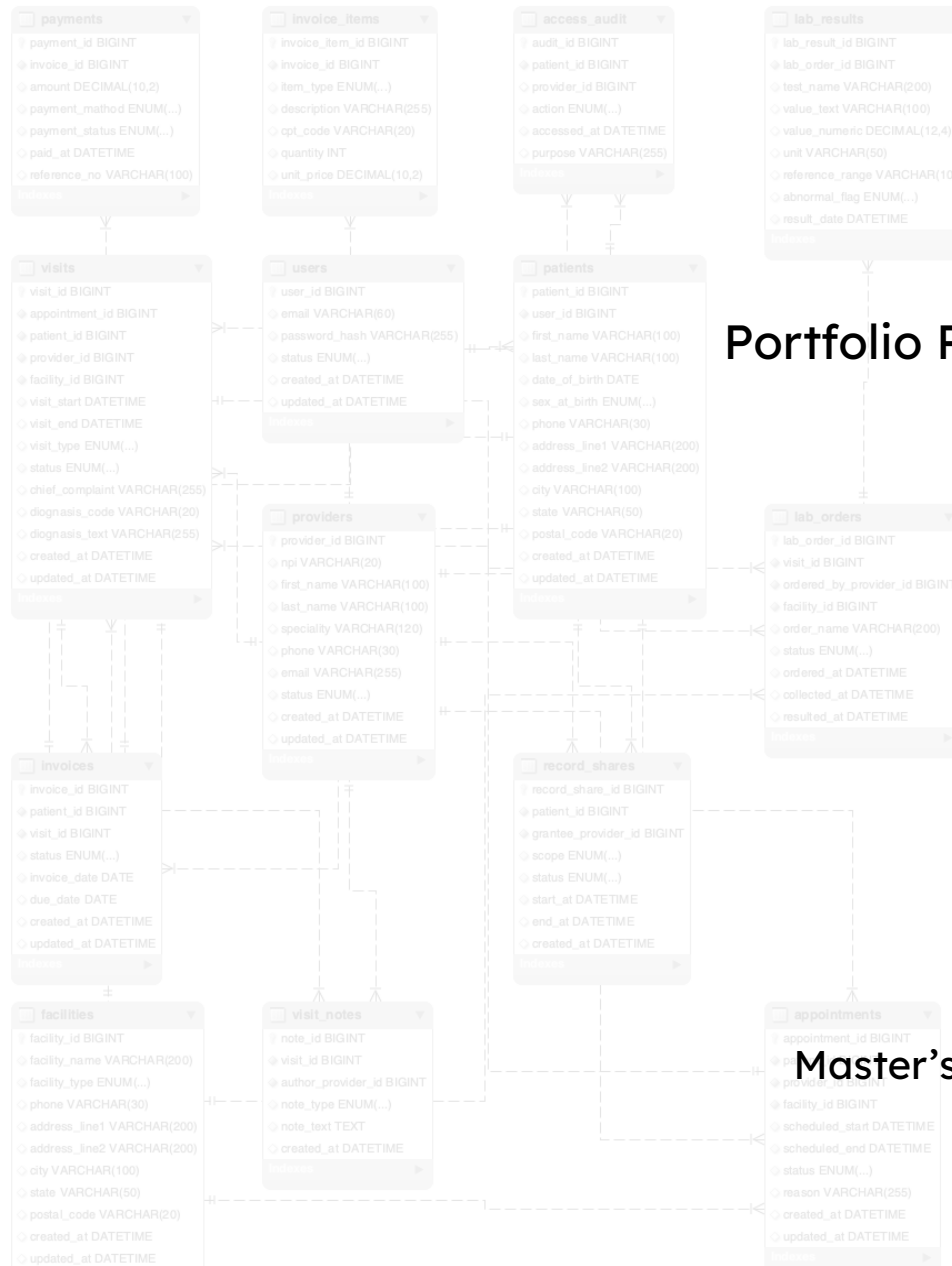


Design and Implementation of a Healthcare Relational Database System



Portfolio Project Report

Maya Tem

Master's in Data Science

April 2026

Index

1. Introduction.....	2
2. Methodology.....	2
2.1 Conceptual Design.....	2
2.2 Schema and Constraints.....	5
3. Database Implementation.....	6
4. Data Population.....	7
5. Validation and Query Analysis.....	9
5.1 Relationship Validation.....	9
5.2 Clinical Data Validation.....	10
5.3 Administrative and Operational Validation.....	11
5.4 Security and Access Control Validation.....	12
5.5 Financial and Billing Validation.....	13
5.6 Summary.....	14
6. Discussion, Conclusion, and Recommendations.....	14
6.1 Discussion and Learning Outcomes.....	14
6.2 Conclusion.....	15
6.3 Limitations.....	15
6.4 Recommendations and Future Work.....	16
Appendix.....	17
Appendix A: Entity-Relationship Diagram.....	17
Appendix B: Database Schema (DDL).....	18
Appendix C: Data Population (DML).....	23

1. Introduction

Healthcare systems rely heavily on structured and well-organized data to support clinical care, administrative operations, and financial processes. In modern healthcare environments, patient portal platforms and digital healthcare systems demonstrate how patients and providers can interact through integrated data-driven solutions. Inspired by these real-world applications, I designed and implemented a relational database system that models a simplified healthcare management platform.

The primary goal of this project was to develop a database that captures the core components of a healthcare system, including patient records, provider interactions, appointments, clinical visits, laboratory processes, and billing activities. Rather than focusing on application development, I concentrated on designing a robust relational schema that ensures data integrity, supports realistic workflows, and reflects how healthcare data is structured and accessed in practice.

My motivation for this project originated from both academic and practical interests. From a technical perspective, I wanted to apply relational database concepts such as normalization, entity relationships, primary and foreign keys, and constraint enforcement in a real-world context. From a practical standpoint, I aimed to understand how complex systems like healthcare platforms manage interconnected data across multiple domains while maintaining consistency and reliability.

The system was designed to support two main perspectives: patients and healthcare providers. Patients can access their medical history, manage appointments, review billing information, and control data-sharing permissions. Providers, on the other hand, can manage patient records, document clinical visits, order laboratory tests, and handle billing processes. By integrating these perspectives into a single data model, the project demonstrates how relational databases can serve as the backbone of modern healthcare systems.

Overall, this project reflects my effort to bridge theoretical knowledge with practical implementation. It highlights not only the technical aspects of database design but also the challenges encountered during development and the decisions required to build a system that is both logically consistent and aligned with real-world requirements.

2. Methodology

2.1 Conceptual Design

Before implementing the database in MySQL, I first focused on designing the system conceptually. At this stage, my goal was to create a clear blueprint that represents how a healthcare platform should organize and manage its data. Instead of jumping directly into coding, I wanted to ensure that the structure of the database was logically consistent and aligned with real-world workflows.

I began by identifying the main entities required in a healthcare system. These included patients, providers, facilities, appointments, visits, lab-related entities, and billing components. Each entity was defined based on its role in the system. For example, the `patients` table stores

demographic and profile information, while the providers table represents clinicians who interact with patients. This separation allowed me to keep responsibilities clear and avoid mixing different types of data.

One important design decision I made early was to separate users from patients. This allowed the system to support different types of users in the future, such as providers or administrators, while keeping authentication data independent from patient-specific information. This approach also made the schema more flexible and scalable.

After defining the entities, I focused on modeling the relationships between them. I carefully analyzed how data flows in a healthcare environment. For example, a patient can have multiple appointments, and each appointment can lead to a visit. A visit can then generate clinical notes, lab orders, and eventually billing records. By following this workflow, I was able to design relationships that reflect actual system behavior rather than just theoretical connections.

Different types of relationships were implemented. One-to-many relationships were used in cases such as patients to visits, where one patient can have multiple visits over time. Many-to-one relationships were used for entities like visit_notes, where multiple notes can belong to a single visit. For more complex interactions, such as controlled data sharing between patients and providers, I used an intermediary table called record_shares to represent a many-to-many relationship.

I also paid attention to how these relationships should behave when data changes. For example, in some cases I used cascading rules to maintain consistency, while in other cases I restricted deletion to preserve important records such as financial transactions or audit logs. These decisions were made based on how real systems prioritize data integrity and traceability.

Another key part of the conceptual design was selecting appropriate attributes and data types for each table. I ensured that each attribute had a clear purpose and avoided redundancy by following normalization principles. For instance, instead of storing repeated information across multiple tables, I used foreign keys to link related data. This not only reduces duplication but also improves consistency across the database.

To further strengthen data integrity, I incorporated constraints such as `NOT NULL`, `UNIQUE`, and `CHECK` conditions. I also used `ENUM` types for fields with controlled values, such as status fields and categories. While this added some restrictions during data insertion, it helped enforce valid data and prevent inconsistencies.

The ER diagram I developed represents the overall structure of the system and visually illustrates the relationships between entities. As shown in Figure 1, the system integrates clinical, administrative, and financial components into a unified model. A more detailed version of the ER diagram is provided in the appendix.

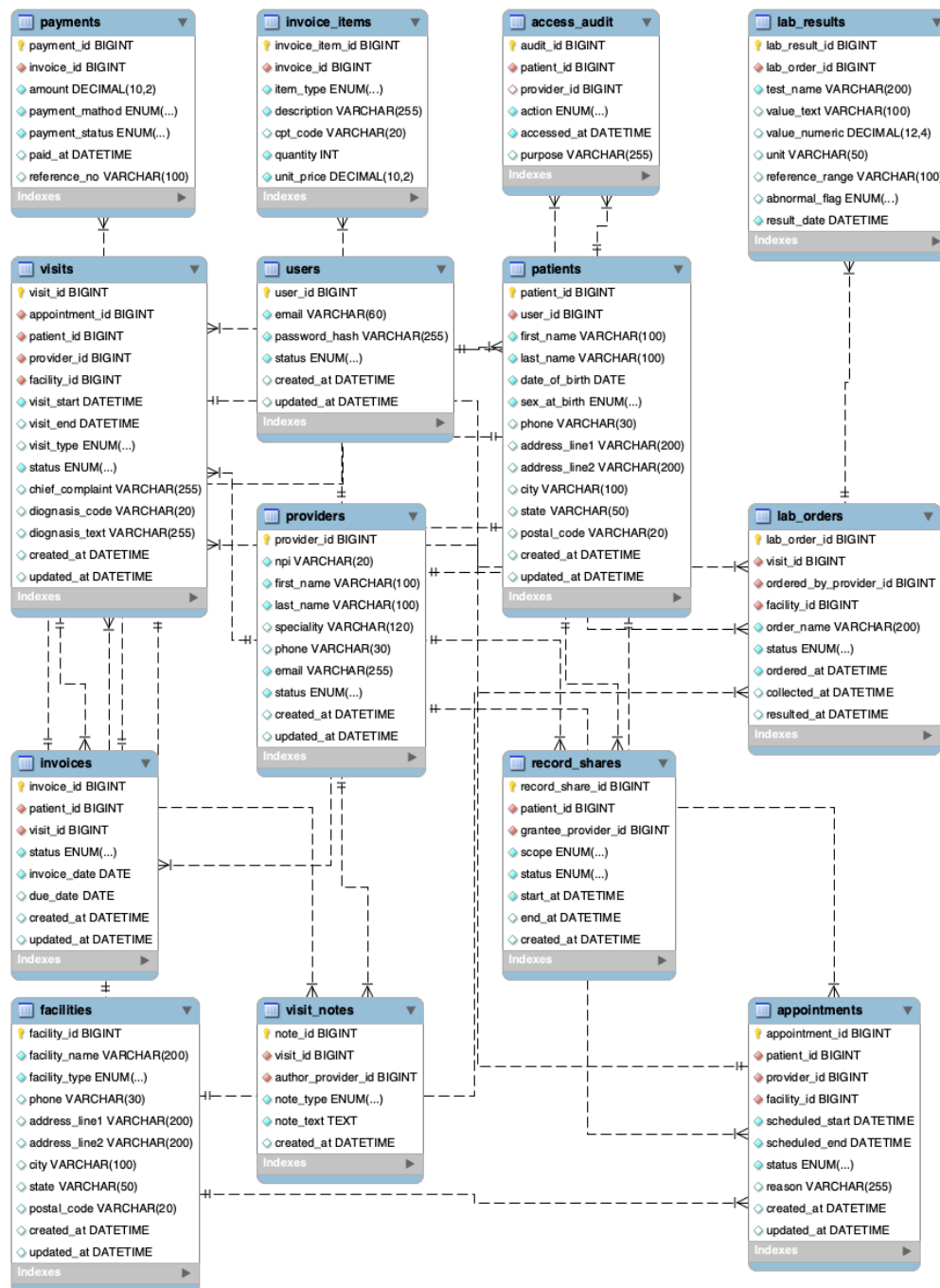


Figure 1: Entity-Relationship Diagram of the Healthcare Database.
 This diagram presents a high-level view of the core entities and their relationships.

Overall, this conceptual design phase was critical for the success of the project. It allowed me to think through the structure of the system, make informed design decisions, and reduce potential issues during implementation. It also gave me a deeper understanding of how relational databases can model complex real-world systems in a structured and efficient way.

2.2 Schema and Constraints

After completing the conceptual design, I translated the structure into a relational schema by defining tables, attributes, and constraints that enforce data integrity within the system. The goal at this stage was not only to represent the entities, but also to ensure that the relationships between them are consistently maintained through the use of primary keys, foreign keys, and domain constraints.

Each table in the database was designed with a primary key to uniquely identify its records. For example, tables such as `patients`, `providers`, `visits`, and `invoices` each use a unique identifier in the form of a `BIGINT` primary key. This ensures that every tuple in the system can be referenced reliably and supports efficient joins across related tables.

Foreign keys were used extensively to establish relationships between tables and to enforce referential integrity. For instance, the `patient_id` in the `visits` table references the `patients` table, while `provider_id` links `visits` to `providers`. Similarly, the `invoices` table is connected to both `patients` and `visits`, ensuring that billing records are always associated with valid clinical encounters. These foreign key relationships prevent orphan records and ensure that all dependent data remains consistent.

In addition to basic key constraints, I applied different referential actions depending on the context of the data. In some cases, I used cascading updates and deletions to automatically maintain consistency. For example, when a parent record is removed, related child records can also be removed if they are no longer meaningful. In contrast, for critical data such as financial transactions and audit logs, I used restrictive constraints to prevent deletion. This ensures that important historical records, such as payments and access logs, are preserved for traceability and accountability.

To further enforce data validity, I incorporated domain constraints such as `NOT NULL`, `UNIQUE`, and `CHECK` conditions. `NOT NULL` constraints were applied to essential attributes such as identifiers, timestamps, and key relationship fields to ensure that required data is always present. `UNIQUE` constraints were used for attributes such as email addresses and provider identifiers to prevent duplication and maintain consistency across the system. `CHECK` constraints were used in specific cases to enforce logical rules, such as ensuring that quantities and monetary values are non-negative.

I also used `ENUM` data types for attributes with a predefined set of valid values. These include fields such as appointment status, visit type, payment method, and access control scope. While `ENUM` constraints required careful alignment between schema definitions and inserted data, they helped prevent invalid entries and ensured that the database maintains consistent categorical values.

Another important design consideration was normalization. I structured the schema to reduce redundancy and avoid storing duplicate information across tables. For example, instead of embedding billing details directly within the `invoices` table, I created a separate `invoice_items` table to represent individual charges. This supports a one-to-many

relationship and reflects how real-world billing systems operate. Similarly, the separation of `lab_orders` and `lab_results` allows the system to handle multiple results for a single test order.

Overall, the schema design and constraint implementation played a critical role in ensuring that the database is both robust and reliable. By carefully defining keys, relationships, and validation rules, I was able to create a system that maintains data integrity while supporting complex healthcare workflows.

3. Database Implementation

After finalizing the conceptual schema and defining the constraints, I proceeded to implement the database using MySQL. This stage involved translating the design into Data Definition Language (DDL) statements to create the database and its constituent tables, while ensuring that all relationships and constraints were properly enforced.

I began by creating the database and selecting it as the working environment. From there, I implemented each table in a logical order, starting with core entities such as `users`, `patients`, `providers`, and `facilities`. These tables do not depend on others and serve as the foundation of the system. Once these base tables were established, I created dependent tables such as `appointments` and `visits`, followed by clinical, financial, and access control tables including `visit_notes`, `lab_orders`, `lab_results`, `invoices`, `invoice_items`, `payments`, `record_shares`, and `access_audit`.

During implementation, I paid close attention to the order of table creation due to foreign key dependencies. Creating tables in the wrong order resulted in constraint errors, which required me to reorganize the sequence to ensure that parent tables were defined before child tables. This experience helped reinforce the importance of dependency management in relational database systems.

Each table was created using appropriate data types and constraints as defined in the design phase. Primary keys were implemented using auto-incrementing `BIGINT` fields to uniquely identify records. Foreign keys were defined with explicit referential actions such as `ON DELETE CASCADE`, `ON DELETE RESTRICT`, and `ON UPDATE CASCADE`, depending on the relationship requirements. These constraints ensured that the integrity of relationships between tables is maintained automatically by the database system. For example, the `visits` table includes foreign key constraints that link it to `patients`, `providers`, and `facilities`, ensuring that each visit is associated with valid and existing records in the system.

```
FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
FOREIGN KEY (provider_id) REFERENCES providers(provider_id);
```

I also incorporated `ENUM` types directly into the schema for attributes with controlled values, such as status fields and categorical data. While implementing these fields, I had to carefully

match the inserted values with the predefined `ENUM` options to avoid errors. This highlighted the importance of consistency between schema definitions and actual data usage.

Throughout the implementation process, I encountered several SQL-related errors that required debugging and schema adjustments. One common issue was related to column naming inconsistencies, which resulted in errors when inserting or referencing data. For example, minor differences in naming conventions caused queries to fail, and I resolved these issues by standardizing column names across all tables.

Another challenge involved data type mismatches and constraint violations, particularly with `ENUM` fields. Attempting to insert values that were not part of the predefined `ENUM` list resulted in errors, which required either correcting the data or adjusting the schema. These situations helped me better understand how strict constraint enforcement contributes to data integrity.

As I progressed, I made iterative improvements to the schema using `ALTER TABLE` statements. These changes included renaming columns, refining constraints, and aligning the schema with the actual use cases I was testing. Rather than viewing these changes as mistakes, I treated them as part of the development process, which ultimately led to a more consistent and reliable design.

In addition to the initial schema design, I applied `ALTER TABLE` operations to modify and extend existing tables. For example, a `created_at` column was added to the `record_shares` table to improve tracking and auditing capabilities. These changes demonstrate how database schemas can evolve during implementation to better support real-world system requirements.

```
ALTER TABLE record_shares
ADD COLUMN created_at DATETIME DEFAULT CURRENT_TIMESTAMP;
```

The complete set of SQL statements used to create the database and tables is provided in the appendix. This includes all DDL scripts with their associated constraints and relationships. By the end of this phase, the database structure was fully implemented and ready for data population and testing.

Overall, the implementation phase allowed me to bridge the gap between theoretical design and practical execution. It provided valuable experience in working with MySQL, handling constraint enforcement, and resolving real-world issues that arise during database development.

4. Data Population

After completing the database implementation, the next step was to populate the tables with sample data. The objective at this stage was not to generate a large dataset, but to create a small yet meaningful set of records that could simulate real-world healthcare workflows and allow the system to be tested effectively.

I approached the data population process incrementally, following the structure of the database and its dependencies. I began with core tables such as `users`, `patients`, `providers`,

and `facilities`, since these entities do not depend on other tables and form the foundation of the system. Once these were populated, I proceeded to insert data into related tables such as `appointments` and `visits`, which represent interactions between `patients` and `providers`.

After establishing these relationships, I continued populating tables related to clinical and operational processes. These included `visit_notes`, `lab_orders`, and `lab_results`. For example, I inserted multiple notes for a single visit to reflect how providers document different aspects of a consultation. Similarly, I created lab orders that produce multiple results, demonstrating a one-to-many relationship between `lab_orders` and `lab_results`.

The financial component of the system was populated next. I inserted records into `invoices`, `invoice_items`, and `payments` to simulate billing workflows. In particular, I ensured that each invoice could contain multiple invoice items, representing different services such as consultations or lab tests. Payments were then linked to `invoices` to reflect scenarios such as full payment, partial payment, or pending balances.

Finally, I populated the `record_shares` and `access_audit` tables to represent data-sharing permissions and access tracking. This allowed me to simulate how patients grant providers access to their records and how the system logs each access event for accountability.

Throughout this process, I paid attention to maintaining consistency across related tables. All foreign key relationships were respected, and records were inserted in a sequence that aligns with the dependency structure of the schema. This ensured that no insertion violated referential integrity constraints.

Rather than inserting random data, I designed the dataset to reflect realistic scenarios. For example, a patient may have multiple appointments, some of which result in completed visits, while others may be cancelled. A single visit may generate clinical notes, lab tests, and billing records. This approach allowed me to test how well the system handles interconnected data across different domains.

In addition to manual data insertion, external data can be imported into the database using CSV files through MySQL's `LOAD DATA INFILE` functionality. This provides an efficient way to populate large datasets in real-world applications.

The SQL statements used for data insertion are provided in the appendix. These include all `INSERT` operations used to populate the database. By the end of this phase, the system contained a coherent dataset that supported meaningful queries and validation.

Overall, the data population stage was essential for transforming the database from a static structure into a functional system. It allowed me to test relationships, verify constraints, and prepare the database for further validation and analysis.

5. Validation and Query Analysis

After populating the database, I developed a series of SQL queries to validate the correctness of the schema, relationships, and overall system behavior. The purpose of this stage was to confirm that the database not only stores data correctly, but also supports meaningful healthcare operations across clinical, administrative, financial, and access control domains.

5.1 Relationship Validation

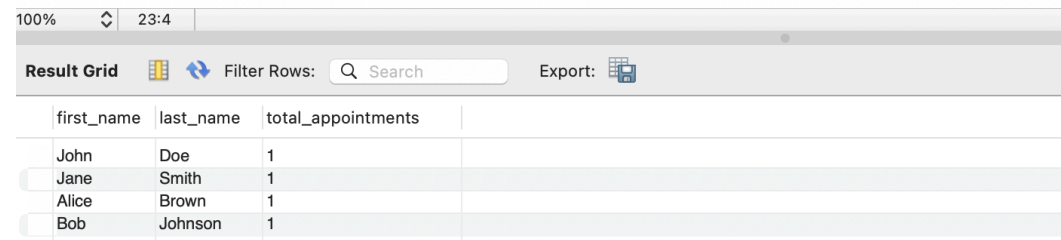
The first stage of validation focused on verifying whether the relationships between tables were implemented correctly. This included testing one-to-one, one-to-many, and many-to-many associations.

To validate the one-to-many relationship between patients and appointments, I wrote a query that counts how many appointments each patient has in the system.

```

1 • SELECT p.first_name, p.last_name, COUNT(a.appointment_id) AS total_appointments
2 FROM patients p
3 JOIN appointments a ON p.patient_id = a.patient_id
4 GROUP BY p.patient_id;

```



first_name	last_name	total_appointments
John	Doe	1
Jane	Smith	1
Alice	Brown	1
Bob	Johnson	1

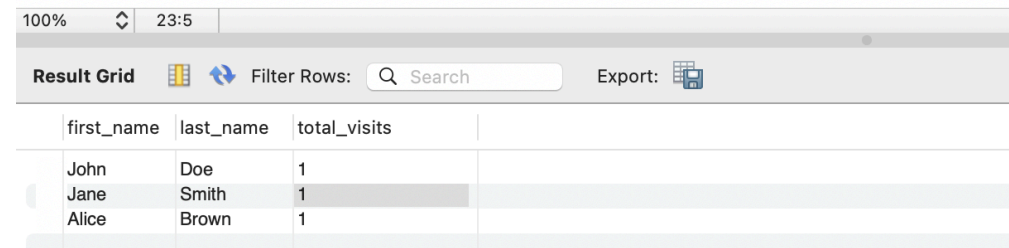
This query confirms that a single patient can be associated with multiple appointments. It also verifies that the foreign key relationship between `appointments` and `patients` is functioning correctly.

To further validate visit activity, I used the following query to count the total number of visits completed by each patient:

```

8 • SELECT p.first_name, p.last_name, COUNT(v.visit_id) AS total_visits
9 FROM patients p
10 JOIN visits v ON p.patient_id = v.patient_id
11 GROUP BY p.patient_id;

```



first_name	last_name	total_visits
John	Doe	1
Jane	Smith	1
Alice	Brown	1

This query confirms that the patient to visit relationship is correctly maintained and that the system can track actual patient encounters over time.

5.2 Clinical Data Validation

I then developed queries to validate the clinical side of the database, especially the connection between patients, visits, notes, and lab workflows.

To verify that visit notes are correctly linked to both the patient and the provider who authored them, I used the following query:

```

13 • SELECT p.first_name, p.last_name, v.visit_start, vn.note_type, vn.note_text,
14        pr.first_name AS provider_name
15 FROM patients p
16 JOIN visits v ON p.patient_id = v.patient_id
17 JOIN visit_notes vn ON v.visit_id = vn.visit_id
18 JOIN providers pr ON vn.author_provider_id = pr.provider_id
19 WHERE p.first_name = 'John' AND p.last_name = 'Doe';

```

00% 16:9

Result Grid Filter Rows: Search Export:

first_name	last_name	visit_start	note_type	note_text	provider_name
John	Doe	2026-04-10 09:05:00	General	Patient presented for annual wellness exam. No acute concerns reported.	Emily
John	Doe	2026-04-10 09:05:00	Prescription	Advised routine preventive care and annual follow-up.	Emily

This query validates that clinical notes are attached to the correct visit and patient. It also confirms that multiple notes can be written for a single visit and that the authoring provider is correctly recorded.

To validate the relationship between lab orders and lab results, I used the following query:

```

22 • SELECT lo.order_name, lo.status AS order_status, lr.test_name,
23        lr.value_numeric, lr.unit, lr.abnormal_flag
24 FROM lab_orders lo
25 JOIN lab_results lr ON lo.lab_order_id = lr.lab_order_id
26 WHERE lr.abnormal_flag = 'Abnormal' OR lo.status = 'Completed';

```

00% 60:18

Result Grid Filter Rows: Search Export:

order_name	order_status	test_name	value_numeric	unit	abnormal_flag
Influenza A/B Test	Completed	Influenza A/B Result	NULL	NULL	Abnormal
Basic Metabolic Panel	Completed	Glucose	102.5000	mg/dL	Abnormal
Basic Metabolic Panel	Completed	Calcium	9.4000	mg/dL	Normal

This query verifies that each lab result is linked to a valid lab order and that abnormal results can be identified correctly. It also demonstrates that one lab order may produce multiple results.

5.3 Administrative and Operational Validation

To assess the operational side of the system, I created queries related to provider activity and facility utilization.

The following query identifies which provider has seen the highest number of unique patients:

```

29 • SELECT pr.provider_id, pr.first_name, pr.last_name,
30       COUNT(DISTINCT v.patient_id) AS total_patients
31 FROM providers pr
32 JOIN visits v ON pr.provider_id = v.provider_id
33 GROUP BY pr.provider_id
34 ORDER BY total_patients DESC;

```

100% 1:27

Result Grid Filter Rows: Search Export:

provider_id	first_name	last_name	total_patients
1	Emily	Clark	2
2	Michael	Lee	1

This query validates the relationship between providers and visits while also showing how the database can support administrative reporting.

To evaluate facility usage, I used the following query:

```

36 • SELECT f.facility_name, f.facility_type,
37       COUNT(a.appointment_id) AS scheduled_appointments
38 FROM facilities f
39 LEFT JOIN appointments a ON f.facility_id = a.facility_id
40 GROUP BY f.facility_id;

```

00% 30:34

Result Grid Filter Rows: Search Export:

facility_name	facility_type	scheduled_appointments
Nashville General Hospital	Clinic	2
Franklin Medical Center	Clinic	2
Tennessee Lab Services	Laboratory	0

This query confirms that appointments are correctly linked to facilities. It also provides useful operational insight by showing how many appointments are scheduled at each location.

5.4 Security and Access Control Validation

The system includes access control features that allow patients to share records with providers under specific permissions. I used queries to validate both active sharing permissions and audit logging.

To determine which providers currently have active access to patient records, I used the following query:

```

54 • SELECT
55     p.first_name AS patient_first,
56     p.last_name AS patient_last,
57     pr.first_name AS provider_first,
58     pr.last_name AS provider_last,
59     rs.scope, rs.end_at
60 FROM record_shares rs
61 JOIN patients p ON rs.patient_id = p.patient_id
62 JOIN providers pr ON rs.grantee_provider_id = pr.provider_id
63 WHERE rs.status = 'Active';

```

100% 61:51

Result Grid Filter Rows: Search Export:

patient_first	patient_last	provider_first	provider_last	scope	end_at
John	Doe	Michael	Lee	Read	2026-05-10 23...
Jane	Smith	Emily	Clark	Full	NULL

This query validates the many-to-many relationship between patients and providers through the `record_shares` table. It also confirms that access permissions and scope values are being stored correctly.

To verify that access events are recorded for traceability, I used the following audit query:

```

66 • SELECT pr.last_name AS provider_name,
67     pa.last_name AS patient_name,
68     aa.action, aa.accessed_at, aa.purpose
69 FROM access_audit aa
70 JOIN providers pr ON aa.provider_id = pr.provider_id
71 JOIN patients pa ON aa.patient_id = pa.patient_id
72 ORDER BY aa.accessed_at DESC
73 LIMIT 4;

```

00% 48:61

Result Grid Filter Rows: Search Export:

provider_name	patient_name	action	accessed_at	purpose
Lee	Brown	View	2026-04-11 12:30:00	Reviewed blood pressure follow-up
Clark	Smith	Update	2026-04-10 11:00:00	Updated treatment plan
Lee	Smith	Create	2026-04-10 10:20:00	Created consultation note
Lee	Doe	View	2026-04-10 10:15:00	Second opinion review

This query demonstrates that the system can track who accessed patient data, when the access occurred, and for what purpose. This is an important feature for accountability and healthcare compliance.

5.5 Financial and Billing Validation

The final stage of validation focused on billing, invoice calculations, and payment tracking.

To calculate outstanding balances for invoices that are still pending or partially paid, I used the following query:

```
75 • SELECT i.invoice_id, p.last_name AS patient_name,
76        SUM(ii.quantity * ii.unit_price) AS total_charge,
77        IFNULL(SUM(pay.amount), 0) AS total_paid,
78        SUM(ii.quantity * ii.unit_price) - IFNULL(SUM(pay.amount), 0) AS balance_due
79 FROM invoices i
80 JOIN patients p ON i.patient_id = p.patient_id
81 JOIN invoice_items ii ON i.invoice_id = ii.invoice_id
82 LEFT JOIN payments pay ON i.invoice_id = pay.invoice_id
83 WHERE i.status IN ('Pending', 'Partially Paid')
84 GROUP BY i.invoice_id;
```

invoice_id	patient_name	total_charge	total_paid	balance_due
1	Doe	120.00	0.00	120.00
3	Brown	95.00	50.00	45.00

This query validates that the financial relationships between `invoices`, `invoice_items`, and `payments` are correctly implemented. It also confirms that the system can calculate remaining balances accurately.

To verify invoice totals based on line items, I used this query:

```
86 • SELECT invoice_id, SUM(quantity * unit_price) AS total_amount
87 FROM invoice_items
88 GROUP BY invoice_id;
```

invoice_id	total_amount
1	120.00
2	230.00
3	95.00

This query confirms that the one-to-many relationship between invoices and invoice items works correctly and that the total value of each invoice can be derived from its detailed charges.

Finally, to obtain a patient-level financial summary, I used the following query:


```

90 • SELECT p.patient_id, p.first_name, p.last_name,
91        SUM(ii.quantity * ii.unit_price) AS total_billed,
92        SUM(pay.amount) AS total_paid
93 FROM patients p
94 JOIN invoices i ON p.patient_id = i.patient_id
95 JOIN invoice_items ii ON i.invoice_id = ii.invoice_id
96 LEFT JOIN payments pay ON i.invoice_id = pay.invoice_id
97 GROUP BY p.patient_id;

```

100% 21:88

Result Grid Filter Rows: Search Export:

	patient_id	first_name	last_name	total_billed	total_paid
1	John	Doe	120.00	NULL	
2	Jane	Smith	230.00	460.00	
3	Alice	Brown	95.00	50.00	

This query provides a cumulative financial view for each patient. It demonstrates that the database can aggregate billing and payment history across multiple encounters and invoices.

5.6 Summary

Overall, the validation process confirmed that the database is logically consistent and operationally reliable. The queries showed that the relationships between tables are correctly enforced, the constraints support valid data entry, and the system can answer practical healthcare questions across multiple domains.

This phase was especially valuable because it moved the project beyond table creation and data insertion. By testing the database through queries, I was able to evaluate how well the system performs in realistic usage scenarios and confirm that the design decisions made earlier were effective.

6. Discussion, Conclusion, and Recommendations

6.1 Discussion and Learning Outcomes

This project provided a comprehensive experience in designing and implementing a relational database system within a real-world domain. Throughout the process, I was able to apply theoretical database concepts and translate them into a functional and structured system.

One of the most important things I learned was how critical the design phase is. Spending time on the conceptual model and relationships before implementation helped reduce major issues later. It allowed me to clearly understand how entities interact and how data flows through the system. This made the implementation phase more manageable and structured.

During the development process, I encountered several challenges that required debugging and iterative refinement. One of the most common issues was related to column naming

inconsistencies, which resulted in SQL errors when inserting or querying data. For example, small differences in naming led to errors such as unknown column references. I addressed this by standardizing naming conventions across all tables.

Another challenge involved `ENUM` constraints. In some cases, I attempted to insert values that did not exactly match the predefined `ENUM` options, which resulted in data truncation errors. Resolving this required careful alignment between schema definitions and inserted data. This experience helped me understand how strict constraints can improve data integrity, but also require precision during implementation.

I also learned the importance of table creation order due to foreign key dependencies. Attempting to create dependent tables before their parent tables resulted in constraint errors. Adjusting the creation sequence reinforced my understanding of referential integrity and dependency management in relational databases.

Overall, these challenges were valuable learning opportunities. Instead of treating them as mistakes, I used them to refine the schema and improve the system. This iterative process helped me build a more reliable and consistent database.

6.2 Conclusion

In this project, I designed and implemented a relational database system that models a simplified healthcare platform. The system integrates clinical, administrative, financial, and access control components into a single structured model.

The database successfully supports key healthcare workflows, including appointment scheduling, visit tracking, clinical documentation, laboratory processing, billing, and controlled data sharing. Through the use of primary keys, foreign keys, and constraints, the system maintains data integrity and ensures consistency across all entities.

The validation queries demonstrated that the database can support meaningful operations and answer practical questions related to patient activity, provider performance, financial status, and data access. This confirms that the system is not only structurally correct but also functionally useful.

This project reflects my ability to design a normalized relational schema, implement it using SQL, and validate it through realistic scenarios. It also demonstrates my understanding of how database systems can be applied to real-world problems.

6.3 Limitations

Although the system provides a solid foundation, there are several limitations that could be addressed in future work.

First, the database does not include an insurance and claims management component. In real healthcare systems, billing is closely tied to insurance providers, and this aspect is not currently modeled.

Second, the system has limited support for handling multiple diagnoses or complex clinical scenarios within a single visit. Extending the schema to support more detailed medical records would improve its realism.

Third, prescription and medication tracking are not included. This is an important feature in healthcare systems and could be added as an extension.

Additionally, while the record sharing mechanism allows patients to grant access to providers, it is relatively simple and could be expanded to support more flexible consent management.

Finally, the project focuses only on the database layer and does not include an application interface, authentication mechanisms, or full compliance features.

6.4 Recommendations and Future Work

Based on the current design, several improvements can be made to enhance the system.

One important extension would be the addition of an insurance and claims module. This would allow the system to better reflect real-world billing processes and support interactions with insurance providers.

Another improvement would be the introduction of a prescriptions table to manage medications prescribed during visits. This would expand the clinical capabilities of the system.

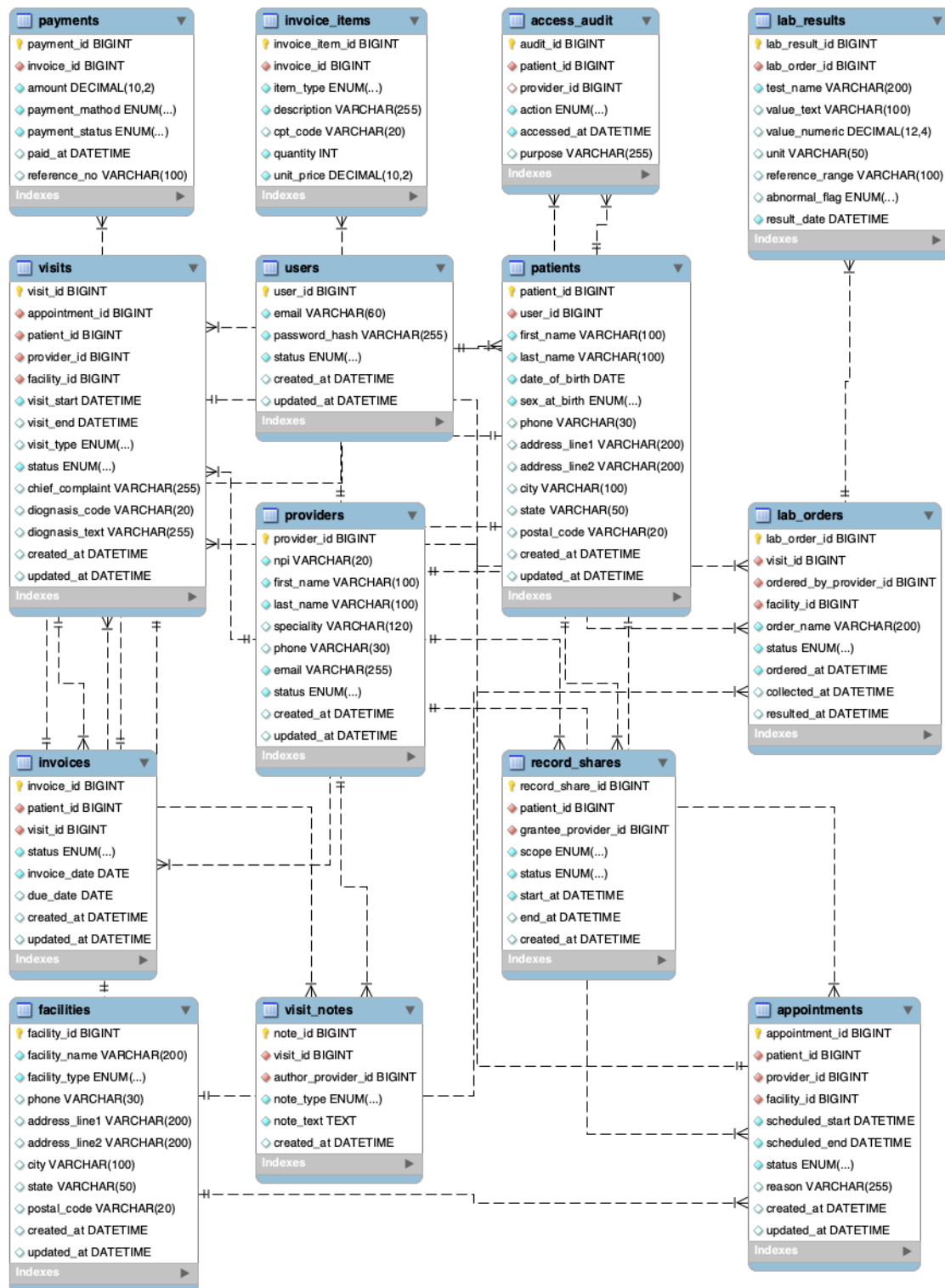
The access control model could also be enhanced by separating consent management from data sharing. This would provide more flexibility and allow more control over who can access specific types of data.

In addition, the system could be extended by adding support for document storage, such as lab reports or medical images. This would make the database more comprehensive and aligned with modern healthcare platforms.

From a technical perspective, the database could be integrated with an application layer to provide user interfaces for patients and providers. This would allow the system to move beyond a backend design and become a complete functional platform.

Appendix

Appendix A: Entity-Relationship Diagram



Appendix B: Database Schema (DDL)

```
#Part 1: database
CREATE DATABASE healthcare_systems;
USE healthcare_systems;

# CREATING TABLES DDL CREATE TABLE, CREATE DATABASE, ALTER , DROP TAABLE
# 1 user table
CREATE TABLE users (
    user_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(60) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    status ENUM('Active', 'Inactive', 'Locked') NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

# 2 patients table
CREATE TABLE patients (
    patient_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    sex_at_birth ENUM('Male', 'Female', 'Other') NOT NULL,
    phone VARCHAR(30),
    address_line1 VARCHAR(200),
    address_line2 VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(50),
    postal_code VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)
        REFERENCES users (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);

# 3 providers table - Strong Entity, parent type, referenced table -
CREATE TABLE providers (
    provider_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    npi VARCHAR(20) NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    speciality VARCHAR(120),
    phone VARCHAR(30),
    email VARCHAR(255) NOT NULL UNIQUE,
    status ENUM('Active', 'Inactive') NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE providers
ADD COLUMN provider_type ENUM('Doctor', 'Nurse', 'Lab Tech', 'Therapist') NOT NULL;
ALTER TABLE providers
RENAME COLUMN speciality TO specialty;

# 4 facilities table Laboratory
CREATE TABLE facilities (
    facility_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    facility_name VARCHAR(200) NOT NULL,
    facility_type ENUM('Hospital', 'Clinic', 'Laboratory', 'Urgent Care') NOT NULL,
    phone VARCHAR(30),
    address_line1 VARCHAR(200),
    address_line2 VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(50),
    postal_code VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE facilities
MODIFY facility_type ENUM('Hospital', 'Clinic', 'Laboratory', 'Urgent Care');
```

```

#Part 2: appointments + visits + visit_notes
# 5 appointments table
CREATE TABLE appointments (
  appointment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  patient_id BIGINT NOT NULL,
  provider_id BIGINT NOT NULL,
  facility_id BIGINT NOT NULL,
  scheduled_start DATETIME NOT NULL,
  scheduled_end DATETIME NOT NULL,
  status ENUM('Scheduled', 'Completed', 'Cancelled', 'No Show') NOT NULL,
  reason VARCHAR(255),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id)
    REFERENCES patients (patient_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (provider_id)
    REFERENCES providers (provider_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (facility_id)
    REFERENCES facilities (facility_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK (scheduled_end > scheduled_start)
);

# 6 visit table
CREATE TABLE visits (
  visit_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  appointment_id BIGINT NOT NULL UNIQUE,
  patient_id BIGINT NOT NULL,
  provider_id BIGINT NOT NULL,
  facility_id BIGINT NOT NULL,
  visit_start DATETIME NOT NULL,
  visit_end DATETIME,
  visit_type ENUM('In-Person', 'Virtual', 'Emergency'),
  status ENUM('Completed', 'Cancelled', 'In Progress') NOT NULL,
  chief_complaint VARCHAR(255),
  diagnosis_code VARCHAR(20),
  diagnosis_text VARCHAR(255),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (appointment_id)
    REFERENCES appointments (appointment_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (patient_id)
    REFERENCES patients (patient_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (provider_id)
    REFERENCES providers (provider_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (facility_id)
    REFERENCES facilities (facility_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK (visit_end IS NULL
    OR visit_end >= visit_start)
);

ALTER TABLE visits
RENAME COLUMN diagnosis_code TO diagnosis_code;
ALTER TABLE visits
RENAME COLUMN diagnosis_text TO diagnosis_text;

```

```

# 7 visit_notes table
CREATE TABLE visit_notes (
    note_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    visit_id BIGINT NOT NULL,
    author_provider_id BIGINT NOT NULL,
    note_type ENUM('Diagnosis', 'Observation', 'Prescription', 'General') NOT NULL,
    note_text TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visit_id)
        REFERENCES visits (visit_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (author_provider_id)
        REFERENCES providers (provider_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

#Part 3: lab_orders + lab_results

# 8 lab_orders table
CREATE TABLE lab_orders (
    lab_order_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    visit_id BIGINT NOT NULL,
    ordered_by_provider_id BIGINT NOT NULL,
    facility_id BIGINT NOT NULL,
    order_name VARCHAR(200) NOT NULL,
    status ENUM('Ordered', 'Collected', 'In Progress', 'Completed', 'Cancelled') NOT NULL,
    ordered_at DATETIME NOT NULL,
    collected_at DATETIME NULL,
    resulted_at DATETIME NULL,
    FOREIGN KEY (visit_id)
        REFERENCES visits (visit_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ordered_by_provider_id)
        REFERENCES providers (provider_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (facility_id)
        REFERENCES facilities (facility_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

# 9 lab_results table
CREATE TABLE lab_results (
    lab_result_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    lab_order_id BIGINT NOT NULL,
    test_name VARCHAR(200) NOT NULL,
    value_text VARCHAR(100),
    value_numeric DECIMAL(12, 4),
    unit VARCHAR(50),
    reference_range VARCHAR(100),
    abnormal_flag ENUM('Normal', 'Abnormal'),
    result_date DATETIME NOT NULL,
    FOREIGN KEY (lab_order_id)
        REFERENCES lab_orders (lab_order_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);

```

```
# Part 4: invoices + invoice_items + payments
```

```
# 10 invoices table
```

```
CREATE TABLE invoices (
  invoice_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  patient_id BIGINT NOT NULL,
  visit_id BIGINT NOT NULL UNIQUE,
  status ENUM('Pending', 'Partially Paid', 'Paid', 'Cancelled') NOT NULL,
  invoice_date DATE NOT NULL,
  due_date DATE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id)
    REFERENCES patients (patient_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (visit_id)
    REFERENCES visits (visit_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
ALTER TABLE invoices
MODIFY status ENUM('Issued', 'Pending', 'Partially Paid', 'Paid', 'Cancelled') NOT NULL;
```

```
# 11 invoice_items table
```

```
CREATE TABLE invoice_items (
  invoice_item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  invoice_id BIGINT NOT NULL,
  item_type ENUM('Consultation', 'Lab Test', 'Procedure', 'Medication', 'Other') NOT NULL,
  description VARCHAR(255) NOT NULL,
  cpt_code VARCHAR(20),
  quantity INT NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL,
  FOREIGN KEY (invoice_id)
    REFERENCES invoices (invoice_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CHECK (quantity > 0),
  CHECK (unit_price >= 0)
);
```

```
# 12 payments table
```

```
CREATE TABLE payments (
  payment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  invoice_id BIGINT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  payment_method ENUM('Cash', 'Credit Card', 'Debit Card', 'Insurance', 'Online') NOT NULL,
  payment_status ENUM('Pending', 'Completed', 'Failed', 'Refunded') NOT NULL,
  paid_at DATETIME NULL,
  reference_no VARCHAR(100) UNIQUE,
  FOREIGN KEY (invoice_id)
    REFERENCES invoices (invoice_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK (amount > 0)
);
```

```
ALTER TABLE payments
RENAME COLUMN payment_method TO payment_method;
```

#Part 5: record_shares + access_audit

13 record_shares table

#access control / permission table

```
CREATE TABLE record_shares (
  record_share_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  patient_id BIGINT NOT NULL,
  grantee_provider_id BIGINT NOT NULL,
  scope ENUM('Read', 'Write', 'Full') NOT NULL,
  status ENUM('Active', 'Revoked', 'Expired') NOT NULL,
  start_at DATETIME NOT NULL,
  end_at DATETIME NULL,
  FOREIGN KEY (patient_id)
    REFERENCES patients (patient_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (grantee_provider_id)
    REFERENCES providers (provider_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  UNIQUE (patient_id , grantee_provider_id),
  CHECK (end_at IS NULL OR end_at >= start_at)
);
```

```
ALTER TABLE record_shares
```

```
ADD COLUMN created_at DATETIME DEFAULT CURRENT_TIMESTAMP
AFTER end_at;
```

14 access_audit table

```
CREATE TABLE access_audit (
  audit_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  patient_id BIGINT NOT NULL,
  provider_id BIGINT NULL,
  action ENUM('View', 'Update', 'Delete', 'Create') NOT NULL,
  accessed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  purpose VARCHAR(255),
  FOREIGN KEY (patient_id)
    REFERENCES patients (patient_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (provider_id)
    REFERENCES providers (provider_id)
    ON DELETE SET NULL ON UPDATE CASCADE
);
```


00%

↕

21:12

Result Grid

Filter Rows:

Edit:

Export/Import:

user_id	email	password_hash	status	created_at	updated_at
1	john.doe@email.com	hashed_pw_1	Active	2026-04-03 13:09:57	2026-04-03 13:09:57
2	jane.smith@email.com	hashed_pw_2	Active	2026-04-03 13:09:57	2026-04-03 13:09:57
3	alice.brown@email.com	hashed_pw_3	Inactive	2026-04-03 13:09:57	2026-04-03 13:09:57
4	bob.johnson@email.com	hashed_pw_4	Active	2026-04-03 13:09:57	2026-04-03 13:09:57
NULL	NULL	NULL	NULL	NULL	NULL

100% 24:24

Result Grid

Filter Rows:

Edit:   

Export/Import:  

	patient_id	user_id	first_name	last_name	date_of_bir...	sex_at_bir...	phone	address_line1	address_line2	city	state	postal_code	created_at	updated_at
1	1	John	Doe	1985-06-15	Male	615-555-1001	123 Main St	NULL		Nashville	TN	37201	2026-04-03 13:12:52	2026-04-03 13:12:52
2	2	Jane	Smith	1990-09-22	Female	615-555-1002	456 Oak Ave	NULL		Nashville	TN	37202	2026-04-03 13:12:52	2026-04-03 13:12:52
3	3	Alice	Brown	1978-03-10	Female	615-555-1003	789 Pine Rd	NULL		Franklin	TN	37064	2026-04-03 13:12:52	2026-04-03 13:12:52
4	4	Bob	Johnson	1982-12-05	Male	615-555-1004	321 Cedar Ln	NULL		Murfreesboro	TN	37130	2026-04-03 13:12:52	2026-04-03 13:12:52
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL		NULL	NULL	NULL	NULL	NULL

[illegible]


```

113 -- #10 invoices table
114 • INSERT INTO invoices
115 (patient_id, visit_id, status, invoice_date, due_date)
116 VALUES
117 (1, 1, 'Pending', '2026-04-10', '2026-04-25'),
118 (2, 2, 'Paid', '2026-04-10', '2026-04-18'),
119 (3, 3, 'Partially Paid', '2026-04-11', '2026-04-26');
120 • SELECT * FROM invoices;

```

100% 1:109

Result Grid Filter Rows: Search Edit: Export/Import:

invoice_id	patient_id	visit_id	status	invoice_date	due_date	created_at	updated_at
1	1	1	Pending	2026-04-10	2026-04-25	2026-04-03 14:25:29	2026-04-03 14:25:29
2	2	2	Paid	2026-04-10	2026-04-18	2026-04-03 14:25:29	2026-04-03 14:25:29
3	3	3	Partially Paid	2026-04-11	2026-04-26	2026-04-03 14:25:29	2026-04-03 14:25:29
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

124 -- #11 invoice_items table
125
126 • INSERT INTO invoice_items
127 (invoice_id, item_type, description, cpt_code, quantity, unit_price)
128 VALUES
129 (1, 'Consultation', 'General wellness visit', '99213', 1, 120.00),
130 (2, 'Consultation', 'Flu visit consultation', '99214', 1, 150.00),
131 (2, 'Lab Test', 'Influenza A/B Test', '87502', 1, 80.00),
132 (3, 'Lab Test', 'Basic Metabolic Panel', '80048', 1, 95.00);
133
134 • SELECT * FROM invoice_items;

```

100% 55:115

Result Grid Filter Rows: Search Edit: Export/Import:

invoice_item...	invoice_id	item_type	description	cpt_code	quantity	unit_price
1	1	Consultation	General wellness visit	99213	1	120.00
2	2	Consultation	Flu visit consultation	99214	1	150.00
3	2	Lab Test	Influenza A/B Test	87502	1	80.00
4	3	Lab Test	Basic Metabolic Panel	80048	1	95.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

136 -- #12 payments table
137 • INSERT INTO payments
138 (invoice_id, amount, payment_method, payment_status, paid_at, reference_no)
139 VALUES
140 (2, 230.00, 'Credit Card', 'Completed', '2026-04-13 10:00:00', 'PAY123456'),
141 (3, 50.00, 'Insurance', 'Pending', NULL, 'INS789101');
142
143 • SELECT * FROM payments;

```

100% 7:128

Result Grid Filter Rows: Search Edit: Export/Import:

payment_id	invoice_id	amount	payment_method	payment_status	paid_at	reference_no
1	2	230.00	Credit Card	Completed	2026-04-13 10:00:00	PAY123456
2	3	50.00	Insurance	Pending	NULL	INS789101
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

145 -- #13 record_shares table
146 • INSERT INTO record_shares
147 (patient_id, grantee_provider_id, scope, status, start_at, end_at, created_at)
148 VALUES
149 (1, 2, 'Read', 'Active', '2026-04-10 08:00:00', '2026-05-10 23:59:59', '2026-04-10 08:00:00'),
150 (2, 1, 'Full', 'Active', '2026-04-10 09:00:00', NULL, '2026-04-10 09:00:00'),
151 (3, 2, 'Write', 'Revoked', '2026-04-11 10:00:00', '2026-04-15 17:00:00', '2026-04-11 10:00:00');
152 • SELECT * FROM record_shares;

```

100% 29:134

Result Grid Filter Rows: Search Edit: Export/Import:

record_share...	patient_id	grantee_provider_id	scope	status	start_at	end_at	created_at
1	1	2	Read	Active	2026-04-10 08:...	2026-05-10 23:...	2026-04-10 08:00:00
2	2	1	Full	Active	2026-04-10 09:...	NULL	2026-04-10 09:00:00
3	3	2	Write	Revoked	2026-04-11 10:...	2026-04-15 17:...	2026-04-11 10:00:00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

154 -- #14 access_audit table
155 • INSERT INTO access_audit
156 (patient_id, provider_id, action, accessed_at, purpose)
157 VALUES
158 (1, 2, 'View', '2026-04-10 10:15:00', 'Second opinion review'),
159 (2, 1, 'Update', '2026-04-10 11:00:00', 'Updated treatment plan'),
160 (3, 2, 'View', '2026-04-11 12:30:00', 'Reviewed blood pressure follow-up'),
161 (2, 2, 'Create', '2026-04-10 10:20:00', 'Created consultation note');
162
163 • SELECT * FROM access_audit;

```

100% 29:152

Result Grid Filter Rows: Search Edit: Export/Import:

audit_id	patient_id	provider_id	action	accessed_at	purpose
1	1	2	View	2026-04-10 10:15:00	Second opinion review
2	2	1	Update	2026-04-10 11:00:00	Updated treatment plan
3	3	2	View	2026-04-11 12:30:00	Reviewed blood pressure follow-up
4	2	2	Create	2026-04-10 10:20:00	Created consultation note
NULL	NULL	NULL	NULL	NULL	NULL